Documentazione BDD - UninaFoodLab

Giuseppe Curi, Ludovica I. Cafasso

2024-2025

UninaFoodLab è un sistema che permette la gestione, da parte di docenti-chef, di un insieme di corsi fruibili da studenti interessati alle varie lezioni culinarie proposte in due metodologie precise: presenza o online; ogni sessione, a prescindere dalla tipologia, può includere la preparazione o presentazione di ricette diverse, scelte dallo chef sulla base del corso di cui saranno protagoniste.

Sono di seguito riportati gli schemi alla base della progettazione del database *UninaFoodLab*; e l'ulteriore documentazione.

Docente: M. Sangiovanni

Progettazione schema della base di dati

Si tratta della prima fase di creazione di un database. Il suo obiettivo non è pensare a "come" memorizzare i dati tecnicamente, ma "cosa" rappresentare del mondo reale.

Diagramma UML non ristrutturato e ER

Note sulla progettazione:

1. La scelta della composizione pone la necessitá di una sessione di avere un proprio corso di riferimento.
2. "*Iscritto*" è una tabella associativa pensata per indicare la data di iscrizione di uno studente ad un determinato corso, mantendendo la gestione di molteplici corsi.
3. La presenza della tabella associativa "*Necessita*" è importante per considerare "*Ingrediente*" un elemento indipendente dalla propria quantitá.
4. L'enumerazione presenta l'uso di XOR, rappresentatato dalla freccia, così da permettere la scelta tra tipologia di unitá di misura o campo vuoto, pensato per prodotti solitamente venduti come unitá.

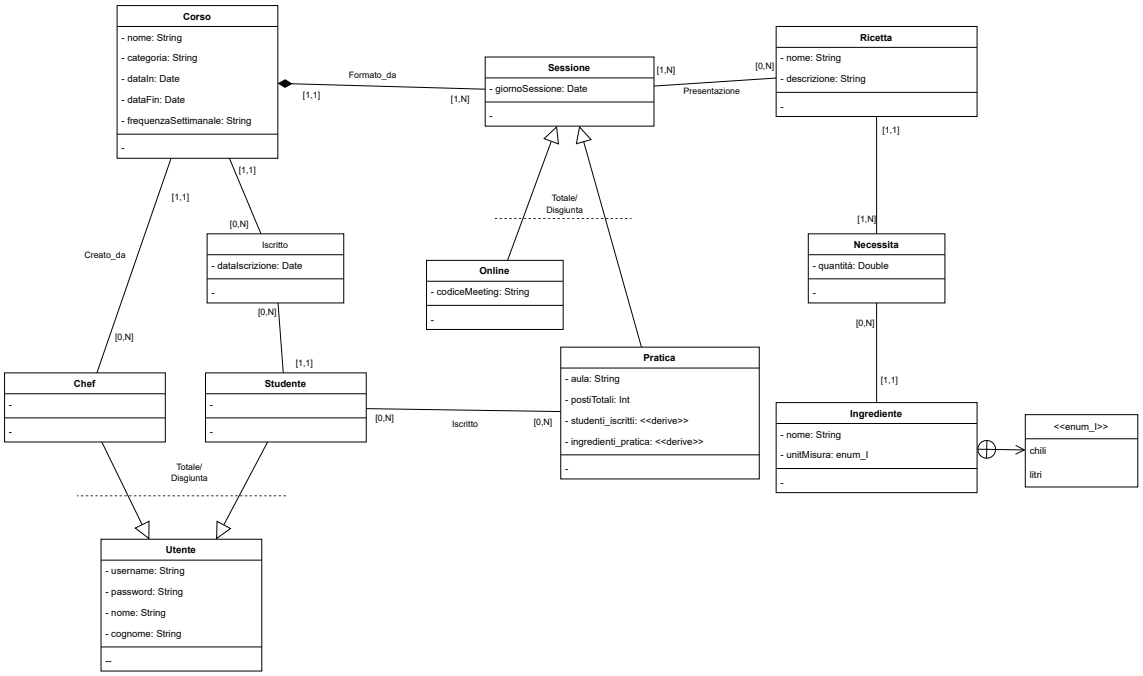

Figure 1: Diagramma UML non ristrutturato

Note entità-relazione:

1. Alcune entitá hanno delle chiavi giá distinguibili(come *Username* sta ad *Utente*) in altri casi abbiamo invece bisogno di ricorrere a quella che chiamiamo chiave surrogata, cioè una chiave avente una keyword "*id*" non naturalmente presente.

2. Gli elementi rappresentati con punto nero rappresentano gli attributi della relazione a cui sono collegati, cioè appartengono all'unione piuttosto che ai singoli.

3. Tra *Corso* e *Sessione* esiste una relazione identificante per l'entitá debole *Sessione*, avente un doppio rettangolo. Questo perché una sessione non può esistere senza il proprio corso di riferimento, la dipendenza è ulteriormente sottolineata dal doppio rombo.

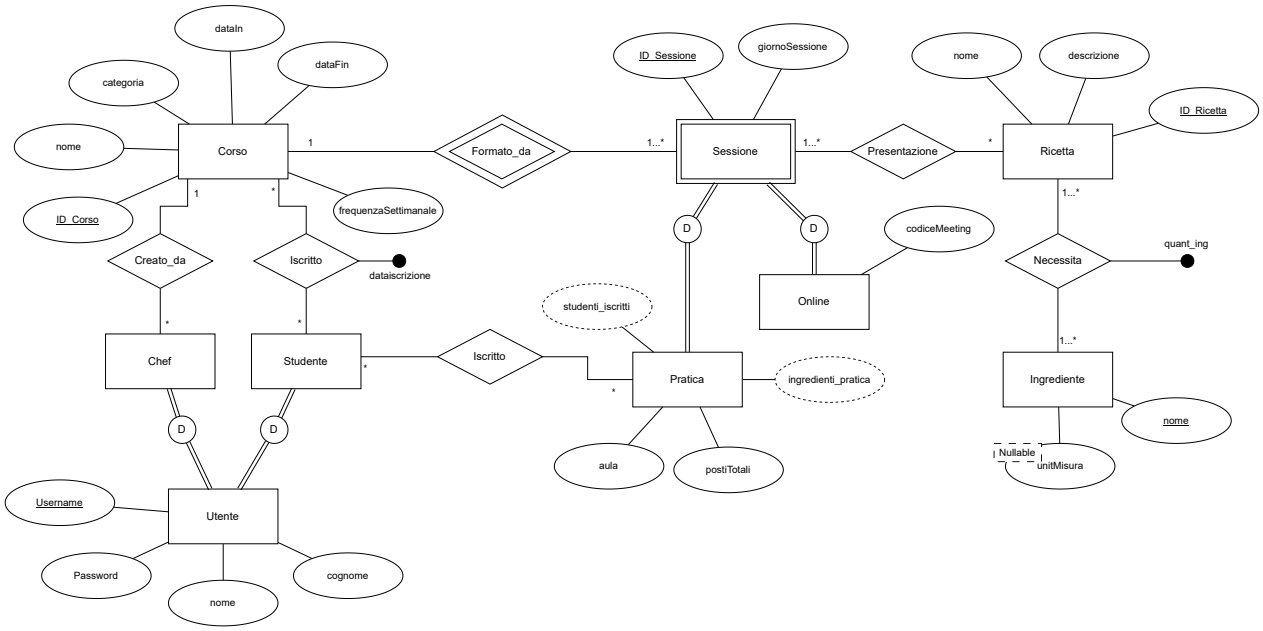

Figure 2: Diagramma Entità-Relazione

Diagramma UML ristrutturato

Il primo passo verso la fase di progettazione logica è la ristrutturazione del nostro UML per adattare uno schema pensato per la comprensione del "problema" ad uno strumento più utile alla produzione del database e più lontano dallo schema concettuale.

Note sulle principali modifiche:

1. Le associazioni molti-a-molti presentano ora ognuna la propria tabella ponte, ovvero un nuovo schema contenente gli attributi dell'associazione e le chiavi per ciascun componente coinvolto.
2. La relazione tra *Corso-Online-Pratica*(ex *Corso-Sessione,ora differenziata*) è ora chiarita grazie all'uso di un legame di aggregazione che permette di mantenere l'idea di dipendenza di *Sessione* adattandolo alle nuove tabelle "*Online-Pratica*".
3. In *Utente* è stata applicata la strategia Single-table Inherutance, introducendo l'attributo *tipo_utente* di tipo enumerativo, che permette la distinzione tra Chefs e Studenti , al fine di evitare ridondanza e mantenere compatibilità con il modello relazionale.

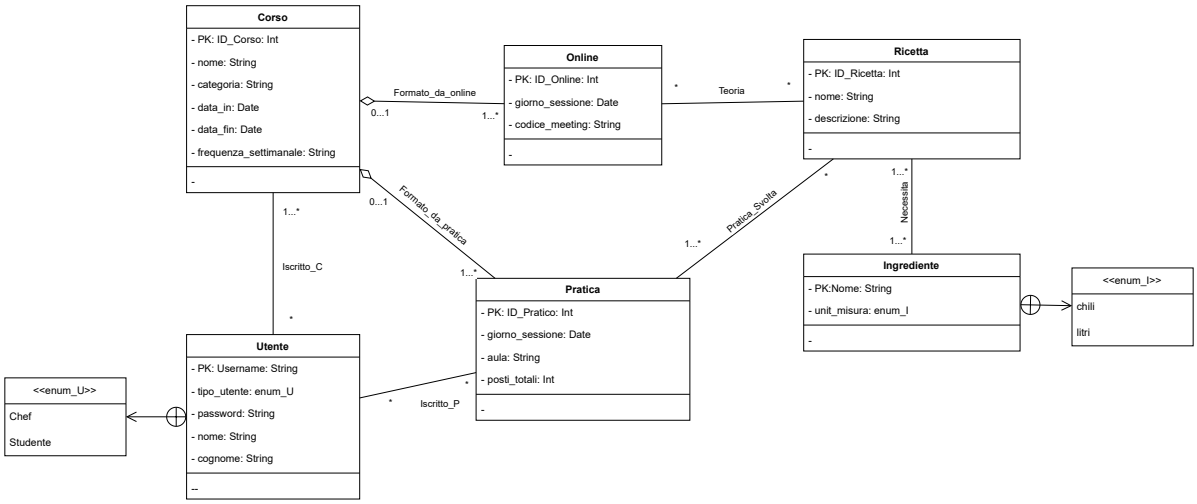

Figure 3: Diagramma UML Ristrutturato

Dizionario delle Classi

0.1 Utente

Registro di tutti gli utenti del sistema; un utente può essere Chef o Studente, e dunque ottenere differenti permessi. Entrambi presentano i medesimi campi attributo.

Attributo	Cardinalità	Descrizione
Username	[1]	Identificativo univoco scelto da un utente per l'accesso
Tipo_utente	[1]	Ruolo assegnato all'utente che può essere "Studente" o "Chef"
Password	[1]	Codice di accesso
Nome	[1]	Nome di battesimo dell'utente registrato
Cognome	[1]	Cognome dell'utente registrato

Table 1: Dizionario delle Classi - Utente

0.2 Corso

Catalogo di tutti i corsi di cucina disponibili nel database, con le relative informazioni riguardo a: tipo di corso, date, frequenze e chef tutor.

Attributo	Cardinalità	Descrizione
Corso_id	[1]	Identificativo numerico univoco del corso generato automaticamente
Nome	[1]	Titolo descrittivo del corso
Categoria	[0,1]	Tipologia culinaria del corso
Data_in	[1]	Data inizio lezioni
Data_fin	[1]	Data di conclusione delle lezioni
Frequenza_settimanale	[0,1]	Indicazione sulla frequenza delle lezioni
Chef_id	[1]	Riferimento allo chef responsabile del corso

Table 2: Dizionario delle Classi - Corso

0.3 Pratica

Registro di tutte le sessioni pratiche associate ai corsi nel sistema; una sessione pratica è un'attività di formazione legata a un corso specifico, con una data, possibilmente un'aula assegnata e un numero massimo di partecipanti.

Attributo	Cardinalità	Descrizione
Id_pratica	[1]	Identificativo univoco della sessione pratica generato automaticamente
Giorno_sessione	[1]	Data in cui si svolge la sessione pratica
Aula	[0,1]	Numero o nome dell'aula in cui si tiene la sessione pratica
Posti_totali	[1]	Numero massimo di posti disponibili per la sessione pratica
Corso_id	[1]	Identificativo univoco del corso a cui è associata la sessione pratica

Table 3: Dizionario delle Classi - Pratica

0.4 Online

Registro di tutte le sessioni online associate ai corsi; una sessione online è un'attività di formazione a distanza legata a un corso specifico, con una data di svolgimento e possibilitá di inserire un codice meeting per l'accesso dei partecipanti.

Attributo	Cardinalità	Descrizione
Id_online	[1]	Identificativo univoco della sessione online generato automaticamente
Giorno_sessione	[1]	Data in cui si svolge la sessione online
Codice_meeting	[0,1]	Codice di accesso o link univoco della sessione online
Corso_id	[1]	Identificativo univoco del corso a cui è associata la sessione online

Table 4: Dizionario delle Classi - Online

0.5 Ricetta

Registro di tutte le ricette presenti nel sistema; una ricetta può contenere informazioni generali su un piatto culinario, oltre al suo nome e all'identificativo.

Attributo	Cardinalità	Descrizione
Id_ricetta	[1]	Identificativo univoco della ricetta generato automaticamente
Nome	[1]	Nome della ricetta
Descrizione	[0,1]	Descrizione dettagliata della ricetta

Table 5: Dizionario delle Classi - Ricetta

0.6 Ingrediente

Elenco degli ingredienti utilizzabili nelle ricette e le relative unitá di misura utili alla preparazione.

Attributo	Cardinalità	Descrizione
Nome	[1]	Identificativo univoco dell'ingrediente
Unit_misura	[0,1]	Enumerativo per identificare il tipo di unità usata per i vari ingredienti

Table 6: Dizionario delle Classi - Ingrediente

1 Dizionario delle Associazioni

1.1 Formato_da_online

Strettamente legato a "corso", ogni corso prevede necessariamente una o più sessioni online svolte su piattaforme in remoto.

Corso[0,1]	Online[1,*]

Table 7: Dizionario delle Associazioni - Formato_da_online

1.2 Formato_da_pratica

Strettamente legato a "corso", ogni corso prevede necessariamente una o più sessioni pratiche svolte in presenza.

Corso[0,1]	Pratica[1,*]

Table 8: Dizionario delle Associazioni - Formato_da_pratica

1.3 Iscritto_C

Permette l'iscrizione di un utente a uno o più corsi; ogni corso può avere molte iscrizioni, in oltre presenta un attributo che permette di salvarne la data di iscrizione.

Corso[1,*]	Utente[0,*]

Table 9: Dizionario delle Associazioni - Iscritto_C

1.4 Iscritto_P

Permette l'iscrizione di un utente a più sessioni pratiche; una sessione pratica può avere diversi iscritti.

Pratica[0,*]	Utente[0,*]

Table 10: Dizionario delle Associazioni - Iscritto_P

1.5 Teoria

Durante le sessioni online si considera la possibilità di presentare una o più ricette che verranno poi possibilmente preparate durante le successive sessioni pratiche.

Online[0,*]	Ricetta[0,*]

Table 11: Dizionario delle Associazioni - Teoria

1.6 Pratica_Svolta

Una sessione pratica si basa necessariamente su una o più ricette, mentre una ricetta potrebbe essere svolta in molteplici sessioni pratiche.

Pratica[1,*]	Ricetta[0,*]

Table 12: Dizionario delle Associazioni - Pratica_Svolta

1.7 Necessita

Ogni ricetta ha bisogno di uno o più ingredienti per essere preparata, in oltre presenta un attributo che permette di salvarne la quantità dell'ingrediente.

Ricetta[1,*]	Ingrediente[1,*]

Table 13: Dizionario delle Associazioni - Necessita

2 Dizionario dei Vincoli

2.1 Utente

Nome Vincolo	Descrizione	Tipologia
`password_is_strong()`	Controlla che la password sia almeno di 6 caratteri.	Dominio

Table 14: Dizionario dei Vincoli - Utente

2.2 Corso

Nome Vincolo	Descrizione	Tipologia
`enforce_chef_id_is_chef()`	Verifica che chef_id punti a un utente di tipo 'Chef'.	Interrelazionale
`enforce_corso_data_in_is_minor_than_data_fin()`	Controlla che data_in (inizio corso) sia minore o uguale di data_fin (fine corso).	Tupla
`enforce_unique_nome_corso_attivo()`	Verifica che il nome del corso non sia già in uso da un corso attivo.	Unicità
`enforce_corso_data_in_not_past()`	Evita che la data inizio corso sia precedente alla data corrente.	Dominio

Table 15: Dizionario dei Vincoli - Corso

2.3 Pratica

Nome Vincolo	Descrizione	Tipologia
`enforce_pratica_checks()`	Verifica che corso_id punti a un corso esistente nella tabella corso.	Integritá Ref.
`prevent_pratica_date_conflict()`	Evita che due sessioni pratiche dello stesso corso abbiano lo stesso giorno_sessione.	Intrarelazionale
`prevent_pratica_online_date_conflict()`	Evita che una sessione pratica e una sessione online dello stesso corso abbiano lo stesso giorno_sessione.	Interrelazionale
`prevent_negative_posti_totali()`	Evita l'inserimento di un numero di posti_totali negativo.	Dominio
`prevent_posti_totali_with_null_aula()`	Previene l'inserimento di un numero di posti_totali diverso da 0 se aula è NULL.	Tupla
`enforce_sessione_date_within_corso()`	Vieta l'inserimento di una sessione al di fuori delle date di inizio e fine corso.	Interrelazionale

Table 16: Dizionario dei Vincoli - Pratica

2.4 Ricetta

Nome Vincolo	Descrizione	Tipologia
`enforce_ricetta_name_is_unique()`	Garantisce che il nome della ricetta sia univoco nel database.	Unicità

Table 17: Dizionario dei Vincoli - Ricetta

2.5 Ingrediente

Nome Vincolo	Descrizione	Tipologia
`check_ingrediente_nome_valido()`	Assicura che il nome dell'ingrediente sia valido (contiene solo lettere, no numeri, simboli, spazi o vuoto).	Dominio

Table 18: Dizionario dei Vincoli - Ingrediente

2.6 Online

Nome Vincolo	Descrizione	Tipologia
`enforce_online_checks()`	Verifica che corso_id punti a un corso esistente nella tabella corso.	Integritá Ref.
`prevent_online_date_conflict()`	Evita che due sessioni online dello stesso corso abbiano lo stesso giorno_sessione.	Intrarelazionale
`prevent_online_pratica_date_conflict()`	Evita che una sessione online e una sessione pratica dello stesso corso abbiano lo stesso giorno_sessione.	Interrelazionale
`enforce_sessione_date_within_corso()`	Vieta l'inserimento di una sessione al di fuori delle date di inizio e fine corso.	Interrelazionale

Table 19: Dizionario dei Vincoli - Online

2.7 Iscritto_P

Nome Vincolo	Descrizione	Tipologia
`enforce_iscrittop_checks()`	Verifica che stud_id sia un utente di tipo 'Studente' e che pratica_id punti a una pratica esistente.	Integritá Ref.
`enforce_max_iscritti_pratica()`	Vieta l'aggiunta di un nuovo iscritto se la pratica ha raggiunto il numero massimo di partecipanti.	Interrelazionale
`enforce_student_enrolled_in_corso()`	Verifica che lo studente sia iscritto al corso prima di iscriversi a una pratica di quel corso.	Integritá Ref.
`no_enroll_finished_pratica()`	Impedisce l'iscrizione a pratiche terminate (uno studente può iscriversi fino un giorno prima della fine).	Interrelazionale
`no_enroll_pratica_before_corso_start()`	Verifica che online_id punti a una sessione online esistente e ricetta_id punti a una ricetta esistente.	Integrità Ref.
`prevent_duplicate_iscrittop()`	Impedisce l'iscrizione doppia di uno studente a una stessa pratica.	Unicità

Table 20: Dizionario dei Vincoli - Iscritto_P

2.8 Iscritto_C

Nome Vincolo	Descrizione	Tipologia
`enforce_iscrittoc_checks()`	Verifica che stud_id sia un utente di tipo "Studente" e che corso_id punti a un corso esistente.	Integritá Ref.
`enforce_iscrittoc_unenroll_checks()`	Controlla se uno studente può disiscriversi da un corso (non deve essere iscritto a pratiche non ancora terminate).	Interrelazionale
`enforce_enroll_corso_date_checks()`	Controlla se è possibile iscriversi al corso - il bando di iscrizione apre una settimana prima dell'inizio e chiude quando il corso inizia.	Interrelazionale
`prevent_duplicate_iscrittoc()`	Impedisce l'iscrizione doppia di uno studente a uno stesso corso.	Unicità

Table 21: Dizionario dei Vincoli - Iscritto_C

2.9 Necessita

Nome Vincolo	Descrizione	Tipologia
`enforce_necessita_checks()`	Verifica che ricetta_id punti a una ricetta esistente e ingrediente_id punti a un ingrediente esistente.	Integritá Ref.
`enforce_positive_quant_ing()`	Controlla che la quantità dell'ingrediente sia sempre positiva.	Dominio
`enforce_non_integer_for_null_unit_misura()`	Impedisce l'inserimento di valori non interi per ingredienti senza unità di misura (pezzi).	Tupla
`prevent_duplicate_Necessita()`	Controlla se una ricetta ha al suo interno già salvato un ingrediente con quel nome.	Unicità

Table 22: Dizionario dei Vincoli - Necessita

2.10 Pratica_Svolta

Nome Vincolo	Descrizione	Tipologia
`enforce_pratica_svolta_checks()`	Verifica che pratica_id punti a una sessione pratica esistente e ricetta_id punti a una ricetta esistente.	Integrità Ref.
`check_ricetta_if_presentata()`	Vieta l'inserimento di una ricetta non svolta in una sessione online precedente alla pratica.	Interrelazionale
`prevent_duplicate_ricetta_in_pratica()`	Impedisce l'iscrizione doppia di una stessa ricetta in una pratica.	Unicità

Table 23: Dizionario dei Vincoli - Pratica_Svolta

2.11 Ponte_Teoria

Nome Vincolo	Descrizione	Tipologia
`enforce_teoria_checks()`	Verifica che online_id punti a una sessione online esistente e ricetta_id punti a una ricetta esistente.	Integrità Ref.
`prevent_duplicate_ricetta_in_online()`	Impedisce l'iscrizione doppia di una stessa ricetta in una sessione online.	Unicità

Table 24: Dizionario dei Vincoli - Ponte_Teoria

Schema logico

La grafica sotto riportata presenta evidenziate in ROSSO le chiavi primarie, in VERDE le esterne, collegate tra di loro dalla propria freccia.

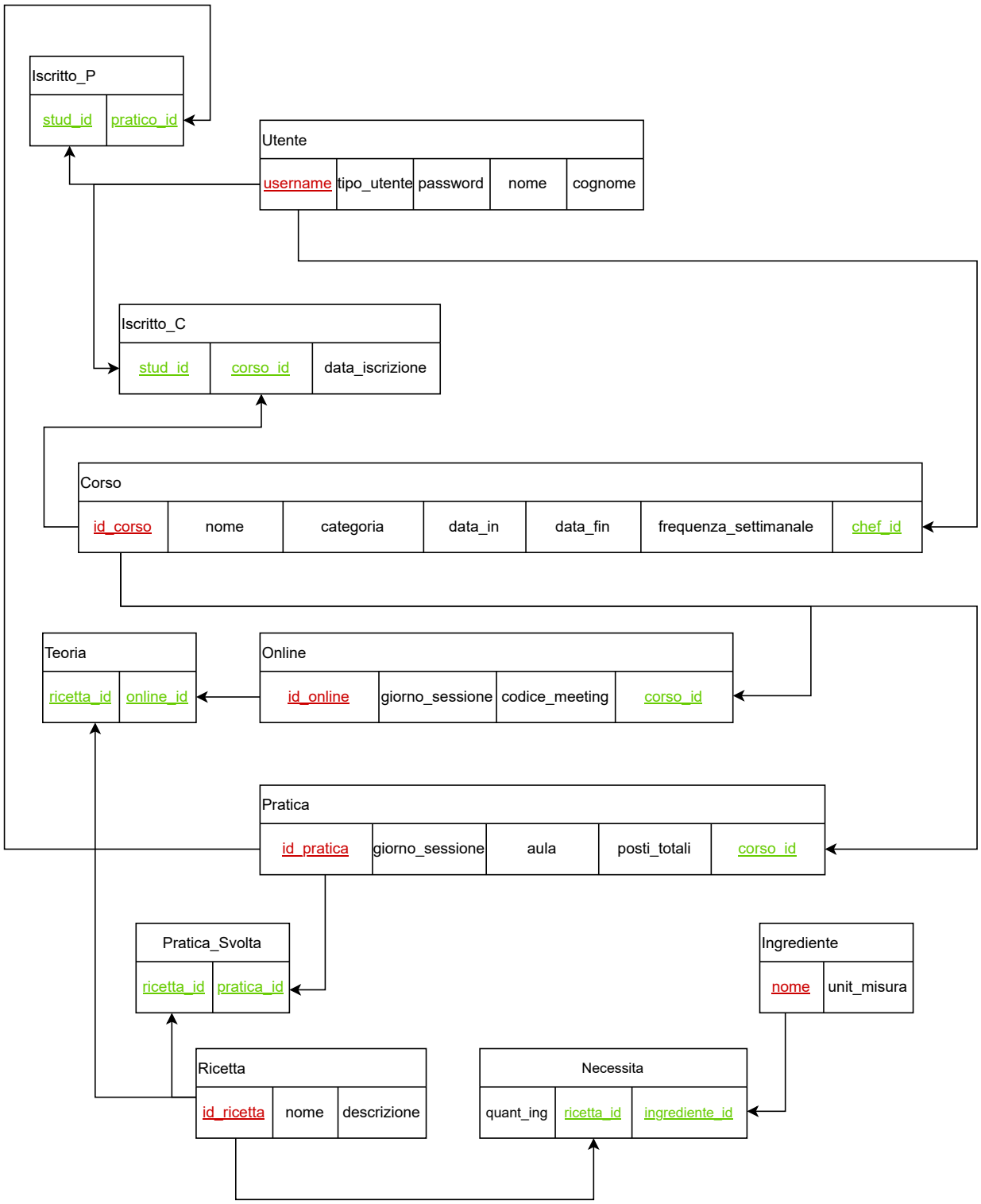

Figure 4: Schema Logico

3 SQL

A seguire il codice SQL realizzato usando la sintassi PostgreSQL; sono stati inseriti alcuni controlli replicano quelli delle foreign key perlo scopo di fornire messaggi di errore più descrittivi rispetto agli errori generici del DBMS.

3.1 Tabelle

3.1.1 Tabella Utente

Listing 1: CreaTabellaUtente.sql

```sql
DO $$ BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'enum_u') THEN
        CREATE TYPE enum_u AS ENUM ('Chef','Studente');
    END IF;
END$$;

CREATE TABLE utente (
    username VARCHAR(100) NOT NULL PRIMARY KEY,
    tipo_utente enum_u NOT NULL DEFAULT 'Studente',
    password VARCHAR(255) NOT NULL,
    nome VARCHAR(100) NOT NULL,
    cognome VARCHAR(100) NOT NULL
);
```

3.1.2 Tabella Corso

Listing 2: CreaTabellaCorso.sql

```sql
CREATE TABLE corso (
    id_corso SERIAL PRIMARY KEY,
    nome VARCHAR(200) NOT NULL,
    categoria VARCHAR(100),
    data_in DATE NOT NULL,
    data_fin DATE NOT NULL,
    frequenza_settimanale VARCHAR(100),
    chef_id VARCHAR(100) NOT NULL
        REFERENCES utente(username) ON UPDATE CASCADE ON DELETE RESTRICT
);
```

3.1.3 Tabella Iscritto_C

Listing 3: CreaTabellaPonteIscritto_C.sql

```sql
CREATE TABLE iscritto_c (
    stud_id VARCHAR(100) NOT NULL,
    corso_id INT NOT NULL,
    data_iscrizione TIMESTAMP(0) DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_iscritto_c PRIMARY KEY (stud_id, corso_id),
    CONSTRAINT fk_studente
        FOREIGN KEY (stud_id)
            REFERENCES utente(username) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_corso
        FOREIGN KEY (corso_id)
            REFERENCES corso(id_corso) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.1.4 Tabella Online

Listing 4: CreaTabellaOnline.sql

```sql
CREATE TABLE online (
    id_online SERIAL PRIMARY KEY,
    giorno_sessione DATE NOT NULL,
    codice_meeting VARCHAR(20),
    corso_id INTEGER NOT NULL,

    CONSTRAINT fk_corso
        FOREIGN KEY(corso_id)
            REFERENCES corso(id_corso) ON DELETE CASCADE
);
```

3.1.5 Tabella Teoria

Listing 5: CreaPonteTeoria.sql

```sql
    CREATE TABLE teoria (
    online_id INT NOT NULL,
    ricetta_id INT NOT NULL,

    CONSTRAINT pk_teoria PRIMARY KEY (online_id, ricetta_id),
    CONSTRAINT fk_online
        FOREIGN KEY(online_id)
            REFERENCES online(id_online) ON DELETE CASCADE ON UPDATE CASCADE,

    CONSTRAINT fk_ricetta_teoria
        FOREIGN KEY(ricetta_id)
            REFERENCES ricetta(id_ricetta) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.1.6 Tabella Pratica

Listing 6: CreaTabellaPratica.sql

```sql
CREATE TABLE pratica (
    id_pratica SERIAL PRIMARY KEY,
    giorno_sessione DATE NOT NULL,
    aula VARCHAR(50),
    posti_totali INTEGER NOT NULL,
    corso_id INTEGER NOT NULL,

    CONSTRAINT fk_corso
        FOREIGN KEY(corso_id)
            REFERENCES corso(id_corso) ON DELETE CASCADE
);
```

3.1.7 Tabella Iscritto_P

Listing 7: CreaTabellaPonteIscritto_P.sql

```sql
CREATE TABLE iscritto_p (
    stud_id VARCHAR(100) NOT NULL,
    pratica_id INT NOT NULL,
    CONSTRAINT pk_iscritto_p PRIMARY KEY (stud_id, pratica_id),
    CONSTRAINT fk_studente
        FOREIGN KEY (stud_id)
            REFERENCES utente(username) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_pratica
        FOREIGN KEY (pratica_id)
            REFERENCES pratica(id_pratica) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.1.8 Tabella Pratica_Svolta

Listing 8: CreaPontePratica_Svolta.sql

```sql

CREATE TABLE pratica_svolta
(
    pratica_id INT NOT NULL,
    ricetta_id INT NOT NULL,

    CONSTRAINT pk_pratica_svolta PRIMARY KEY (pratica_id, ricetta_id),
    CONSTRAINT fk_pratica
        FOREIGN KEY(pratica_id)
            REFERENCES pratica(id_pratica) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ricetta
        FOREIGN KEY(ricetta_id)
            REFERENCES ricetta(id_ricetta) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.1.9 Tabella Ricetta

Listing 9: CreaTabellaRicetta.sql

```sql
    CREATE TABLE ricetta (
    id_ricetta SERIAL PRIMARY KEY,
    nome VARCHAR(200) NOT NULL,
    descrizione TEXT
);
```

3.1.10 Tabella Ingredienti

Listing 10: CreaTabellaIngredienti.sql

```sql

    DO $$ BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'enum_i') THEN
        CREATE TYPE enum_i AS ENUM ('chili','litri');
    END IF;
END$$;
CREATE TABLE ingrediente (
    nome VARCHAR(100) NOT NULL PRIMARY KEY,
    unit_misura enum_i
);
```

3.1.11 Tabella Necessita

Listing 11: CreaTabellaNecessita.sql

```sql

CREATE TABLE necessita (
    ricetta_id INT NOT NULL,
    ingrediente_id VARCHAR(100) NOT NULL,
    quant_ing DECIMAL NOT NULL,
    CONSTRAINT pk_necessita PRIMARY KEY (ricetta_id, ingrediente_id),
    CONSTRAINT fk_ricetta
        FOREIGN KEY (ricetta_id)
            REFERENCES ricetta(id_ricetta) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ingrediente
        FOREIGN KEY (ingrediente_id)
            REFERENCES ingrediente(nome) ON DELETE CASCADE ON UPDATE CASCADE
);
```

3.2 Funzioni Trigger

3.2.1 Funzioni Utente

Listing 12: 1.0- FunTabellaUtente.sql

```sql
--Creazione funzione e trigger per controllare la forza della password
CREATE OR REPLACE FUNCTION password_is_strong()
RETURNS TRIGGER AS $$
BEGIN
    IF length(NEW.password)<6 THEN
        RAISE EXCEPTION 'La password deve essere lunga almeno 6 caratteri';
    END IF;
    RETURN NEW;
END;
$$LANGUAGE plpgsql;

CREATE TRIGGER trg_password_is_strong
BEFORE INSERT OR UPDATE ON utente
FOR EACH ROW EXECUTE FUNCTION password_is_strong();
```

3.2.2 Funzioni Corso

Listing 13: 2.0- FunTabellaCorso.sql

```sql
-- Funzione che verifica che chef_id punti a un utente di tipo 'Chef'
CREATE OR REPLACE FUNCTION enforce_chef_id_is_chef()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.chef_id IS NOT NULL THEN
        PERFORM 1 FROM utente WHERE username = NEW.chef_id AND tipo_utente = 'Chef';
        IF NOT FOUND THEN
            RAISE EXCEPTION 'L''utente di nome "%" non e'' uno Chef', NEW.chef_id;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_chef_id_is_chef
BEFORE INSERT OR UPDATE ON corso
FOR EACH ROW EXECUTE FUNCTION enforce_chef_id_is_chef();
```

Listing 14: 2.1- FunTabellaCorso.sql

```sql
--Funzione che controlla che data_in (inizio corso) sia minore di data_fin(fine corso)
CREATE OR REPLACE FUNCTION enforce_corso_data_in_is_minor_than_data_fin()
RETURNS TRIGGER AS $$
BEGIN
        IF NEW.data_in > NEW.data_fin THEN
            RAISE EXCEPTION 'La data d''inizio "%" deve essere minore o uguale alla data di fine "%"', NEW.data_in, NEW.data_fin;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_corso_data_in_is_minor_than_data_fin
BEFORE INSERT OR UPDATE ON corso
FOR EACH ROW EXECUTE FUNCTION enforce_corso_data_in_is_minor_than_data_fin();
```

Listing 15: 2.2- FunTabellaCorso.sql

```sql
-- Funzione che verifica che il nome del corso non sia gia' in uso da un corso attivo dallo stesso chef
CREATE OR REPLACE FUNCTION enforce_unique_nome_corso_attivo()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM 1 FROM corso
    WHERE nome = NEW.nome AND data_fin >= NEW.data_in AND data_in <= NEW.data_fin AND id_corso != NEW.id_corso AND chef_id = NEW.chef_id;

    IF FOUND THEN
        RAISE EXCEPTION 'Esiste gia'' un corso attivo con nome "%". Il nome puo'' essere riutilizzato solo dopo il completamento del corso precedente.', NEW.nome;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_unique_nome_corso_attivo
BEFORE INSERT OR UPDATE ON corso
FOR EACH ROW EXECUTE FUNCTION enforce_unique_nome_corso_attivo();
```

Listing 16: 2.3- FunTabellaCorso.sql

```sql
--Questo trigger evita che la data inizio corso sia precedente alla data corrente
CREATE OR REPLACE FUNCTION enforce_corso_data_in_not_past()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.data_in < CURRENT_DATE THEN
        RAISE EXCEPTION 'La data d''inizio "%" non puo'' essere precedente alla data corrente "%"', NEW.data_in, CURRENT_DATE;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_corso_data_in_not_past
BEFORE INSERT OR UPDATE ON corso
FOR EACH ROW EXECUTE FUNCTION enforce_corso_data_in_not_past();
```

Listing 17: 2.4- FunTabellaCorso.sql

```sql
--trigger che vieta l'inserimento di una sessione al di fuori delle date di inizio e fine corso
CREATE OR REPLACE FUNCTION enforce_sessione_date_within_corso()
RETURNS TRIGGER AS $$
DECLARE
    corso_start_date DATE;
    corso_end_date DATE;
BEGIN
    SELECT data_in, data_fin INTO corso_start_date, corso_end_date FROM corso WHERE id_corso = NEW.corso_id;
    IF corso_start_date IS NULL OR corso_end_date IS NULL THEN
        RAISE EXCEPTION 'Il corso con id "%" non esiste', NEW.corso_id;
    END IF;
    IF NEW.giorno_sessione < corso_start_date OR NEW.giorno_sessione > corso_end_date THEN
        RAISE EXCEPTION 'La data della sessione "%" deve essere compresa tra la data di inizio "%" e la data di fine "%" del corso', NEW.giorno_sessione, corso_start_date, corso_end_date;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

--Trigger definito in tabella pratica
```

3.2.3 Funzioni iscritto_C

Listing 18: 3.0- FunPonteIscritto_C.sql

```sql
-- Funzione che verifica che stud_id punti a un utente di tipo 'Studente' e che corso_id punti a un corso esistente
CREATE OR REPLACE FUNCTION enforce_iscrittoc_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stud_id IS NOT NULL THEN
        PERFORM 1 FROM utente WHERE username = NEW.stud_id AND tipo_utente = 'Studente';
        IF NOT FOUND THEN
            RAISE EXCEPTION 'L''utente "%" non e'' uno Studente', NEW.stud_id;
        END IF;
    END IF;

    IF NEW.corso_id IS NOT NULL THEN
        PERFORM 1 FROM corso WHERE id_corso = NEW.corso_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'Il corso con id "%" non esiste', NEW.corso_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_iscrittoc_checks
BEFORE INSERT OR UPDATE ON iscritto_c
FOR EACH ROW EXECUTE FUNCTION enforce_iscrittoc_checks();
```

Listing 19: 3.1- FunPonteIscritto_C.sql

```sql
-- Trigger che controlla se uno studente puo' disiscriversi da un corso (se non e' iscritto a nessuna pratica associata a quel corso)
--(o se la pratica non e' ancora finita)
CREATE OR REPLACE FUNCTION enforce_iscrittoc_unenroll_checks()
RETURNS TRIGGER AS $$
DECLARE
    pratica_record RECORD;
BEGIN
    FOR pratica_record IN
        SELECT p.id_pratica
        FROM pratica p
        JOIN iscritto_p ip ON p.id_pratica = ip.pratica_id
        WHERE ip.stud_id = OLD.stud_id AND p.corso_id = OLD.corso_id
    LOOP
        IF NOT is_pratica_finished(pratica_record.id_pratica) THEN
            RAISE EXCEPTION 'Lo studente "%" non puo'' disiscriversi dal corso dal nome "%" perche'' e'' iscritto ad almeno una pratica non ancora terminata',
                OLD.stud_id, (SELECT nome FROM corso WHERE id_corso = OLD.corso_id);
        END IF;
    END LOOP;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_enforce_iscrittoc_unenroll_checks
BEFORE DELETE ON iscritto_c
FOR EACH ROW EXECUTE FUNCTION enforce_iscrittoc_unenroll_checks();
```

Listing 20: 3.2- FunPonteIscritto_C.sql

```sql
-- Trigger che controlla se e' possibile iscriversi al corso: il bando di iscrizione apre una settimana prima dell'inizio e chiude quando il corso inizia
CREATE OR REPLACE FUNCTION enforce_enroll_corso_date_checks()
RETURNS TRIGGER AS $$
DECLARE
    corso_start_date DATE;
    corso_end_date DATE;
    enroll_open_date DATE;
    corso_nome VARCHAR;
BEGIN
    -- Recupera i dati del corso
    SELECT data_in, data_fin, nome INTO corso_start_date, corso_end_date, corso_nome FROM corso WHERE id_corso = NEW.corso_id;
    -- Calcola la data di apertura del bando (una settimana prima dell'inizio)
    enroll_open_date := corso_start_date - INTERVAL '7 days';
    -- Controlla se il corso e' gia' terminato usando la funzione is_corso_finished()
    IF is_corso_finished(NEW.corso_id, NEW.data_iscrizione::DATE) THEN
        RAISE EXCEPTION 'Non e'' possibile iscriversi al corso di nome"%" perche'' e'' gia'' terminato il "%".', corso_nome, corso_end_date;
    END IF;
    -- Controlla se la data di iscrizione e' prima dell'apertura del bando
    IF NEW.data_iscrizione::DATE < enroll_open_date THEN
        RAISE EXCEPTION 'Non e'' possibile iscriversi al corso di nome"%" perche'' il bando non e'' ancora aperto. Il bando aprira'' il "%".', corso_nome, enroll_open_date;
    END IF;
    -- Controlla se il corso e' gia' iniziato usando la funzione is_corso_started()
    IF is_corso_started(NEW.corso_id, NEW.data_iscrizione::DATE) THEN
        RAISE EXCEPTION 'Non e'' possibile iscriversi al corso di nome"%" perche'' il corso e'' gia'' iniziato il "%".', corso_nome, corso_start_date;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_enroll_corso_date_checks
BEFORE INSERT ON iscritto_c
FOR EACH ROW EXECUTE FUNCTION enforce_enroll_corso_date_checks();
```

Listing 21: 3.3- FunPonteIscritto_C.sql

```sql
-- Trigger che impedisce l'iscrizione doppia di uno studente a uno stesso corso
CREATE OR REPLACE FUNCTION prevent_duplicate_iscrittoc()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM 1 FROM iscritto_c WHERE stud_id = NEW.stud_id AND corso_id = NEW.corso_id;
    IF FOUND THEN
        RAISE EXCEPTION 'Lo studente "%" e'' gia'' iscritto al corso con nome "%".', NEW.stud_id, (SELECT nome FROM corso WHERE id_corso = NEW.corso_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_duplicate_iscrittoc
BEFORE INSERT ON iscritto_c
FOR EACH ROW EXECUTE FUNCTION prevent_duplicate_iscrittoc();
```

3.2.4 Funzioni Online

Listing 22: 4.0- FunTabellaOnline.sql

```sql
--trigger che verifica che corso_id punti a un corso esistente
CREATE OR REPLACE FUNCTION enforce_online_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.corso_id IS NOT NULL THEN
        PERFORM 1 FROM corso WHERE id_corso = NEW.corso_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'corso_id "%" non corrisponde a un corso esistente', NEW.
                corso_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_online_checks
BEFORE INSERT OR UPDATE ON online
FOR EACH ROW EXECUTE FUNCTION enforce_online_checks();
```

Listing 23: 4.1- FunTabellaOnline.sql

```sql
--trigger per stesso giorno_sessione in online
CREATE OR REPLACE FUNCTION prevent_online_date_conflict()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.giorno_sessione IS NOT NULL AND NEW.corso_id IS NOT NULL THEN
        IF EXISTS (
            SELECT 1 FROM online
            WHERE giorno_sessione = NEW.giorno_sessione AND corso_id = NEW.corso_id AND
                id_online <> NEW.id_online
        ) THEN
            RAISE EXCEPTION 'Conflitto di data: esiste gia'' una sessione online per il
                corso di nome "%" nella data "%" ', (SELECT nome FROM corso WHERE
                id_corso = NEW.corso_id), NEW.giorno_sessione;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_online_date_conflict
BEFORE INSERT OR UPDATE ON online
FOR EACH ROW EXECUTE FUNCTION prevent_online_date_conflict();
```

Listing 24: 4.2-FunTabellaOnline.sql

```sql
--trigger per evitare che sessioni online e pratica abbiano lo stesso giorno_sessione per lo stesso corso
CREATE OR REPLACE FUNCTION prevent_online_pratica_date_conflict()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.giorno_sessione IS NOT NULL AND NEW.corso_id IS NOT NULL THEN
        IF EXISTS (
            SELECT 1 FROM pratica
            WHERE giorno_sessione = NEW.giorno_sessione AND corso_id = NEW.corso_id
        ) THEN
            RAISE EXCEPTION 'Conflitto di data: esiste gia'' una sessione pratica per il corso di nome "%" nella data "%" ', (SELECT nome FROM corso WHERE id_corso = NEW.corso_id), NEW.giorno_sessione;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_online_pratica_date_conflict
BEFORE INSERT OR UPDATE ON online
FOR EACH ROW EXECUTE FUNCTION prevent_online_pratica_date_conflict();
```

Listing 25: 4.3-FunTabellaOnline.sql

```sql
 --Riutilizzo di codice funzione(Trigger) riferirirsi a 1.4 in FunTabellaCorso

CREATE TRIGGER trg_enforce_sessione_date_within_corso
BEFORE INSERT OR UPDATE ON online
FOR EACH ROW EXECUTE FUNCTION enforce_sessione_date_within_corso();
```

3.2.5 Funzioni Teoria

Listing 26: 5.0-FunPonteTeoria.sql

```sql
-- Funzione che verifica che online_id punti a una sessione online esistente e che ricetta_id punti a una ricetta esistente
CREATE OR REPLACE FUNCTION enforce_teoria_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.online_id IS NOT NULL THEN
        PERFORM 1 FROM online WHERE id_online = NEW.online_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La sessione online con id "%" non esiste', NEW.online_id;
        END IF;
    END IF;

    IF NEW.ricetta_id IS NOT NULL THEN
        PERFORM 1 FROM ricetta WHERE id_ricetta = NEW.ricetta_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La ricetta con id "%" non esiste', NEW.ricetta_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_teoria_checks
BEFORE INSERT OR UPDATE ON teoria
FOR EACH ROW EXECUTE FUNCTION enforce_teoria_checks();
```

Listing 27: 5.1-FunPonteTeoria.sql

```sql
-- Trigger che impedisce l'assegnazione doppia di una stessa ricetta in una sessione online
CREATE OR REPLACE FUNCTION prevent_duplicate_ricetta_in_online()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM teoria WHERE online_id = NEW.online_id AND ricetta_id = NEW.ricetta_id
    ) THEN
        RAISE EXCEPTION 'La ricetta con nome "%" e'' gia'' stata assegnata alla sessione online del giorno "%".', (SELECT nome FROM ricetta WHERE id_ricetta = NEW.ricetta_id), (SELECT giorno_sessione FROM online WHERE id_online = NEW.online_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_duplicate_ricetta_in_online
BEFORE INSERT ON teoria
FOR EACH ROW EXECUTE FUNCTION prevent_duplicate_ricetta_in_online();
```

3.2.6 Funzioni Pratica

Listing 28: 6.0- FunTabellaPratica.sql

```sql
--trigger che verifica che corso_id punti a un corso esistente
CREATE OR REPLACE FUNCTION enforce_pratica_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.corso_id IS NOT NULL THEN
        PERFORM 1 FROM corso WHERE id_corso = NEW.corso_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'corso_id "%" non corrisponde a un corso esistente', NEW.
                corso_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_pratica_checks
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION enforce_pratica_checks();
```

Listing 29: 6.1- FunTabellaPratica.sql

```sql
--trigger per check della presenza di una sessione nello stesso giorno_sessione in
    pratica
CREATE OR REPLACE FUNCTION prevent_pratica_date_conflict()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.giorno_sessione IS NOT NULL AND NEW.corso_id IS NOT NULL THEN
        IF EXISTS (
            SELECT 1 FROM pratica
            WHERE giorno_sessione = NEW.giorno_sessione AND corso_id = NEW.corso_id AND
                id_pratica <> NEW.id_pratica
        ) THEN
            RAISE EXCEPTION 'Conflitto di data: esiste gia'' una sessione pratica per il
                 corso di nome "%" nella data "%" ', (SELECT nome FROM corso WHERE
                id_corso = NEW.corso_id), NEW.giorno_sessione;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_pratica_date_conflict
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION prevent_pratica_date_conflict();
```

Listing 30: 6.2- FunTabellaPratica.sql

```sql
--trigger per evitare che sessioni online e pratica abbiano lo stesso giorno_sessione per lo stesso corso
CREATE OR REPLACE FUNCTION prevent_pratica_online_date_conflict()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.giorno_sessione IS NOT NULL AND NEW.corso_id IS NOT NULL THEN
        IF EXISTS (
            SELECT 1 FROM online
            WHERE giorno_sessione = NEW.giorno_sessione AND corso_id = NEW.corso_id
        ) THEN
            RAISE EXCEPTION 'Conflitto di data: esiste gia'' una sessione online per il corso di nome "%" nella data "%" ', (SELECT nome FROM corso WHERE id_corso = NEW.corso_id), NEW.giorno_sessione;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_pratica_online_date_conflict
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION prevent_pratica_online_date_conflict();
```

Listing 31: 6.4- FunTabellaPratica.sql

```sql
-- Riutilizzo di codice funzione ( Trigger ) riferirirsi a 1.4 in FunTabellaCorso

CREATE TRIGGER trg_enforce_sessione_date_within_corso
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION enforce_sessione_date_within_corso();
```

Listing 32: 6.5- FunTabellaPratica.sql

```sql
--Trigger che evita l'inserimento di posti_totali negativi
CREATE OR REPLACE FUNCTION prevent_negative_posti_totali()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.posti_totali < 0 THEN
        RAISE EXCEPTION 'Il numero di posti_totali non puo'' essere negativo: %', NEW.posti_totali;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_negative_posti_totali
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION prevent_negative_posti_totali();
```

Listing 33: 6.6- FunTabellaPratica.sql

```sql
--Trigger che previene l'inserimento di un numero di posti_totali diverso da 0 se aula nulla
CREATE OR REPLACE FUNCTION prevent_posti_totali_with_null_aula()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.aula IS NULL AND NEW.posti_totali <> 0 THEN
        RAISE EXCEPTION 'Se aula e'' impostata al valore NULL, il numero di posti totali deve essere 0, non %', NEW.posti_totali;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_posti_totali_with_null_aula
BEFORE INSERT OR UPDATE ON pratica
FOR EACH ROW EXECUTE FUNCTION prevent_posti_totali_with_null_aula();
```

3.2.7 Funzioni Iscritto_P

Listing 34: 7.0- FunPonteIscritto_P.sql

```sql
-- funzione che verifica che stud_id punti a un utente di tipo 'Studente' e che pratica_id punti a una pratica esistente
CREATE OR REPLACE FUNCTION enforce_iscrittop_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.stud_id IS NOT NULL THEN
        PERFORM 1 FROM utente WHERE username = NEW.stud_id AND tipo_utente = 'Studente';
        IF NOT FOUND THEN
            RAISE EXCEPTION 'L''utente "%" non e'' uno Studente', NEW.stud_id;
        END IF;
    END IF;

    IF NEW.pratica_id IS NOT NULL THEN
        PERFORM 1 FROM pratica WHERE id_pratica = NEW.pratica_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La pratica con id "%" non esiste', NEW.pratica_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_iscrittop_checks
BEFORE INSERT OR UPDATE ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION enforce_iscrittop_checks();
```

Listing 35: 7.1- FunPonteIscritto_P.sql

```sql
--trigger che vieta l'aggiunta di un nuovo iscritto se la pratica ha gia raggiunto il numero massimo di iscritti
-- Utilizza view_pratica_posti per il calcolo dei posti rimanenti
CREATE OR REPLACE FUNCTION enforce_max_iscritti_pratica()
RETURNS TRIGGER AS $$
BEGIN
    IF (SELECT posti_rimanenti FROM view_studenti_iscritti WHERE id_pratica = NEW.pratica_id) <= 0 THEN
        RAISE EXCEPTION 'La pratica del giorno "%" ha raggiunto il numero massimo di iscritti', (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_enforce_max_iscritti_pratica
BEFORE INSERT ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION enforce_max_iscritti_pratica();
```

Listing 36: 7.2- FunPonteIscritto_P.sql

```sql
-- Funzione che verifica che lo studente sia iscritto al corso prima di iscriversi a una pratica di quel corso
CREATE OR REPLACE FUNCTION enforce_student_enrolled_in_corso()
RETURNS TRIGGER AS $$
DECLARE
    corso_id_pratica INT;
BEGIN
    SELECT corso_id INTO corso_id_pratica FROM pratica WHERE id_pratica = NEW.pratica_id;

    PERFORM 1 FROM iscritto_c WHERE stud_id = NEW.stud_id AND corso_id = corso_id_pratica;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Lo studente "%" non e'' iscritto al corso dal nome "%" a cui appartiene la pratica avente data "%". Iscrizione non effettuata',
        NEW.stud_id, (SELECT nome FROM corso WHERE id_corso = corso_id_pratica), (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_student_enrolled_in_corso
BEFORE INSERT OR UPDATE ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION enforce_student_enrolled_in_corso();
```

Listing 37: 7.3- FunPonteIscritto_P.sql

```sql
-- Trigger che impedisce l'iscrizione a pratiche terminate (uno studente puo' iscriversi fino un giorno prima della fine)
CREATE OR REPLACE FUNCTION no_enroll_finished_pratica()
RETURNS TRIGGER AS $$
BEGIN
    IF is_pratica_finished(NEW.pratica_id, (CURRENT_DATE + INTERVAL '1 d')::DATE) THEN
        RAISE EXCEPTION 'Non e'' possibile iscriversi alla pratica del giorno "%" in quanto e'' gia'' terminato il periodo d''iscrizione.', (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_enroll_finished_pratica
BEFORE INSERT OR UPDATE ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION no_enroll_finished_pratica();
```

Listing 38: 7.4- FunPonteIscritto_P.sql

```sql
-- Trigger che impedisce l'iscrizione a pratiche dove il corso non e' ancora iniziato
CREATE OR REPLACE FUNCTION no_enroll_pratica_before_corso_start()
RETURNS TRIGGER AS $$
DECLARE
    corso_id_pratica INT;
BEGIN
    SELECT corso_id INTO corso_id_pratica FROM pratica WHERE id_pratica = NEW.pratica_id;

    IF NOT is_corso_started(corso_id_pratica) THEN
        RAISE EXCEPTION 'Non e'' possibile iscriversi alla pratica del giorno "%" prima dell''inizio del corso previsto per il "%".',
        (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id), (SELECT data_in FROM corso WHERE id_corso = corso_id_pratica);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_no_enroll_pratica_before_corso_start
BEFORE INSERT OR UPDATE ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION no_enroll_pratica_before_corso_start();
```

Listing 39: 7.5- FunPonteIscritto_P.sql

```sql
--Trigger che impedisce l'iscrizione doppia di uno studente a una stessa pratica
CREATE OR REPLACE FUNCTION prevent_duplicate_iscrittop()
RETURNS TRIGGER AS $$
BEGIN
    PERFORM 1 FROM iscritto_p WHERE stud_id = NEW.stud_id AND pratica_id = NEW.pratica_id;
    IF FOUND THEN
        RAISE EXCEPTION 'Lo studente "%" e'' gia'' iscritto alla pratica del giorno "%".', NEW.stud_id, (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_duplicate_iscrittop
BEFORE INSERT ON iscritto_p
FOR EACH ROW EXECUTE FUNCTION prevent_duplicate_iscrittop();
```

3.2.8 Funzioni Pratica_Svolta

Listing 40: 8.0- FunPontePratica_Svolta.sql

```sql
-- Funzione che verifica che pratica_id punti a una sessione pratica esistente e che
    ricetta_id punti a una ricetta esistente
CREATE OR REPLACE FUNCTION enforce_pratica_svolta_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.pratica_id IS NOT NULL THEN
        PERFORM 1 FROM pratica WHERE id_pratica = NEW.pratica_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La pratica con id "%" non corrisponde a una sessione
                pratica esistente', NEW.pratica_id;
        END IF;
    END IF;

    IF NEW.ricetta_id IS NOT NULL THEN
        PERFORM 1 FROM ricetta WHERE id_ricetta = NEW.ricetta_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La ricetta con id "%" non esiste', NEW.ricetta_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_pratica_svolta_checks
BEFORE INSERT OR UPDATE ON pratica_svolta
FOR EACH ROW EXECUTE FUNCTION enforce_pratica_svolta_checks();
```

Listing 41: 8.1- FunPontePratica_Svolta.sql

```sql
--Trigger che vieta l'inserimento di una ricetta che non e' stata svolta in una sessione pratica precedente alla online
CREATE OR REPLACE FUNCTION check_ricetta_if_presentata()
RETURNS TRIGGER AS $$
DECLARE
    corso_pratica INTEGER;
    giorno_pratica DATE;
    giorno_online DATE;
BEGIN
    SELECT corso_id, giorno_sessione INTO corso_pratica, giorno_pratica
    FROM pratica WHERE id_pratica=NEW.pratica_id;

    SELECT onl.giorno_sessione INTO giorno_online
    FROM teoria t JOIN online onl ON t.online_id = onl.id_online
    WHERE t.ricetta_id=NEW.ricetta_id AND onl.corso_id=corso_pratica AND onl.giorno_sessione < giorno_pratica
    ORDER BY onl.giorno_sessione DESC LIMIT 1;

    IF giorno_online IS NULL THEN
        RAISE EXCEPTION 'Il nome della "%" non e'' stata presentata in una sessione online prima della pratica del giorno "%"',(SELECT nome FROM ricetta WHERE id_ricetta = NEW.ricetta_id), (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_ricetta_if_presentata
BEFORE INSERT OR UPDATE ON pratica_svolta
FOR EACH ROW EXECUTE FUNCTION check_ricetta_if_presentata();
```

Listing 42: 8.2- FunPontePratica_Svolta.sql

```sql
-- Trigger che impedisce l'iscrizione doppia di una stessa ricetta in una pratica
CREATE OR REPLACE FUNCTION prevent_duplicate_ricetta_in_pratica()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM pratica_svolta
        WHERE pratica_id = NEW.pratica_id AND ricetta_id = NEW.ricetta_id
    ) THEN
        RAISE EXCEPTION 'La ricetta con nome "%" e'' gia'' stata inserita per la pratica avente data "%"', (SELECT nome FROM ricetta WHERE id_ricetta = NEW.ricetta_id), (SELECT giorno_sessione FROM pratica WHERE id_pratica = NEW.pratica_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_prevent_duplicate_ricetta_in_pratica
BEFORE INSERT ON pratica_svolta
FOR EACH ROW EXECUTE FUNCTION prevent_duplicate_ricetta_in_pratica();
```

3.2.9 Funzioni Ricetta

Listing 43: 9.0-FunTabellaRicetta.sql

```sql
-- Funzione e trigger per garantire che il nome della ricetta sia univoco
CREATE OR REPLACE FUNCTION enforce_ricetta_name_is_unique()
RETURNS TRIGGER AS $$
    BEGIN
        IF NEW.nome IS NOT NULL THEN
            PERFORM 1 FROM ricetta WHERE nome = NEW.nome AND id_ricetta <> NEW.
                id_ricetta;
            IF FOUND THEN
                RAISE EXCEPTION 'Una ricetta con nome "%" esiste gia''', NEW.nome;
            END IF;
        END IF;
        RETURN NEW;
    END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ricetta_name_is_unique
BEFORE INSERT OR UPDATE ON ricetta
FOR EACH ROW EXECUTE FUNCTION enforce_ricetta_name_is_unique();
```

3.2.10 Funzioni Ingrediente

Listing 44: 10.0-FunTabellaIngredienti.sql

```sql
--trigger che assicura che il nome dell'ingrediente sia valido (non numeri,simboli,spazi
    o vuoto)
CREATE OR REPLACE FUNCTION check_ingrediente_nome_valido()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.nome !~ '^[A-Za- z                                                    ]+$' THEN
        RAISE EXCEPTION 'Errore: il nome dell''ingrediente % non e'' valido. Deve
            contenere solo lettere.', NEW.nome;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_ingrediente_nome_valido
BEFORE INSERT OR UPDATE ON ingrediente
FOR EACH ROW EXECUTE FUNCTION check_ingrediente_nome_valido();
```

3.2.11 Funzioni Necessita

Listing 45: 11.0-FunTabellaPonteNecessita.sql

```sql
-- Funzione che verifica che ricetta_id punti a una ricetta esistente e che ingrediente_id punti a un ingrediente esistente
CREATE OR REPLACE FUNCTION enforce_necessita_checks()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.ricetta_id IS NOT NULL THEN
        PERFORM 1 FROM ricetta WHERE id_ricetta = NEW.ricetta_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'La ricetta con id "%" non esiste', NEW.ricetta_id;
        END IF;
    END IF;

    IF NEW.ingrediente_id IS NOT NULL THEN
        PERFORM 1 FROM ingrediente WHERE nome = NEW.ingrediente_id;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'L''ingrediente di nome "%" non esiste', NEW.ingrediente_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_enforce_necessita_checks
BEFORE INSERT OR UPDATE ON necessita
FOR EACH ROW EXECUTE FUNCTION enforce_necessita_checks();
```

Listing 46: 11.1-FunTabellaPonteNecessita.sql

```sql
--Trigger che la quantit  dell'ingrediente sia sempre positiva
CREATE OR REPLACE FUNCTION enforce_positive_quant_ing()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.quant_ing <= 0 THEN
        RAISE EXCEPTION 'La quantita'' dell''ingrediente deve essere positiva e diversa da zero. Valore fornito: %', NEW.quant_ing;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_enforce_positive_quant_ing
BEFORE INSERT OR UPDATE ON necessita
FOR EACH ROW EXECUTE FUNCTION enforce_positive_quant_ing();
```

Listing 47: 11.2-FunTabellaPonteNecessita.sql

```sql
--Trigger che impedice l'inserimento di valori non interi se nell unit_misura di ingrediente e' NULL(un pezzo)
CREATE OR REPLACE FUNCTION enforce_non_integer_for_null_unit_misura()
RETURNS TRIGGER AS $$
DECLARE
    unit_misura_val enum_i;
BEGIN
    SELECT unit_misura INTO unit_misura_val FROM ingrediente WHERE nome = NEW.ingrediente_id;

    IF unit_misura_val IS NULL AND NEW.quant_ing != FLOOR(NEW.quant_ing) THEN
        RAISE EXCEPTION 'Per l''ingrediente "%", la quantita'' deve essere un numero intero. Valore fornito: %', NEW.ingrediente_id, NEW.quant_ing;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_enforce_non_integer_for_null_unit_misura
BEFORE INSERT OR UPDATE ON necessita
FOR EACH ROW EXECUTE FUNCTION enforce_non_integer_for_null_unit_misura();
```

Listing 48: 11.3-FunTabellaPonteNecessita.sql

```sql
--Trigger utile a controllare se una ricetta ha al suo interno gia' salvato un ingrediente con quel nome
CREATE OR REPLACE FUNCTION prevent_duplicate_Necessita()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS(
        SELECT 1 FROM necessita
        WHERE ricetta_id = NEW.ricetta_id AND ingrediente_id = NEW.ingrediente_id
    ) THEN
        RAISE EXCEPTION 'L''ingrediente "%" e' gia' presente nella ricetta con nome %', NEW.ingrediente_id, (SELECT nome FROM ricetta WHERE id_ricetta = NEW.ricetta_id);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_duplicate_Necessita_trigger
BEFORE INSERT OR UPDATE ON necessita
FOR EACH ROW EXECUTE FUNCTION prevent_duplicate_Necessita();
```

4 Funzioni Ausiliarie

4.1 Dizionario delle funzioni ausiliarie

si tratta di frammenti di codice riutilizzabili, pensati per eseguire compiti semplici e specifici che supportano le operazioni principali.

Nome funzione	Parametri	Descrizione	Ritorno
`is_corso_finished()`	`corso_id INT, checkDate DATE DEFAULT CURRENT_DATE`	Restituisce *true* se un corso è finito (data_fin passata), *false* altrimenti.	BOOLEAN
`is_corso_started()`	`corso_id INT, checkDate DATE DEFAULT CURRENT_DATE`	Restituisce *true* se un corso è iniziato (data_in passata), *false* altrimenti.	BOOLEAN
`is_pratica_finished()`	`praticaId INT, checkDate DATE DEFAULT CURRENT_DATE`	Restituisce *true* se una pratica è finita (giorno_sessione passato), *false* altrimenti.	BOOLEAN

Table 25: Dizionario delle Funzioni ausiliarie

4.2 Codice Funzioni Ausiliarie

Listing 49: 2.5- FunTabellaCorso.sql

```
--Funzione che restituisce true se un corso(id) e' finito(data_fin passato), false
    altrimenti(ausiliaria)
CREATE OR REPLACE FUNCTION is_corso_finished(corso_id INT, checkDate DATE DEFAULT
    CURRENT_DATE)
RETURNS BOOLEAN AS $$
DECLARE
    corso_end_date DATE;
BEGIN
    SELECT data_fin INTO corso_end_date FROM corso WHERE id_corso = corso_id;
    IF corso_end_date IS NULL THEN
        RAISE EXCEPTION 'Il corso con id "%" non esiste', corso_id;
    END IF;
    RETURN corso_end_date < checkDate;
END;
$$ LANGUAGE plpgsql;
```

Listing 50: 2.6- FunTabellaCorso.sql

```
--Funzione che restituisce true se un corso(id) e' iniziato(data_in passato), false
    altrimenti
CREATE OR REPLACE FUNCTION is_corso_started(corso_id INT, checkDate DATE DEFAULT
    CURRENT_DATE)
RETURNS BOOLEAN AS $$
DECLARE
    corso_start_date DATE;
BEGIN
    SELECT data_in INTO corso_start_date FROM corso WHERE id_corso = corso_id;
    IF corso_start_date IS NULL THEN
        RAISE EXCEPTION 'Il corso con id "%" non esiste', corso_id;
    END IF;
    RETURN corso_start_date <= checkDate;
END;
$$ LANGUAGE plpgsql;
```

Listing 51: 6.3- FunTabellaPratica.sql

```
--Funzione che restituisce true se una pratica(id)    finita(giorno_sessione passato),
    false altrimenti (ausiliaria)
CREATE OR REPLACE FUNCTION is_pratica_finished(praticaId INT, checkDate DATE DEFAULT
    CURRENT_DATE)
RETURNS BOOLEAN AS $$
DECLARE
    pratica_date DATE;
BEGIN
    SELECT giorno_sessione INTO pratica_date FROM pratica WHERE id_pratica = praticaId;
    IF pratica_date IS NULL THEN
        RAISE EXCEPTION 'La pratica con id "%" non esiste', praticaId;
    END IF;
    RETURN pratica_date < checkDate;
END;
$$ LANGUAGE plpgsql;
```

5 View

L'uso delle view è consigliabile in situazioni, come in queste riportate, in cui si ritiene preferibile ottenere informazioni da tabelle già presenti senza però "gravare" ulteriormente sul database, memorizzando stabilmente ulteriori dati.

5.1 Dizionario delle View

Nome View	`view_studenti_iscritti`
Descrizione	Visualizza il numero di posti totali, occupati e rimanenti per ogni sessione pratica.
Dipendenze	tabella **pratica**, tabella **iscritto_p**.
Utilizzo	Controllo dei posti rimanenti nelle pratiche(presente in `enforce_max_iscritti_pratica` nella tabella Iscritto_P).

Table 26: Dizionario delle View - Studenti Iscritti

Nome View	`view_ingredienti_pratica`
Descrizione	Visualizza il totale di ingredienti utilizzati per ogni pratica (somma degli ingredienti di tutte le ricette associate, moltiplicato per il numero di iscritti).
Dipendenze	tabella **pratica**, tabella **pratica_svolta**, tabella **ricetta**, tabella **necessita**, tabella **ingrediente**, view **view_studenti_iscritti**.
Utilizzo	Calcolo delle quantità di ingredienti necessari per le sessioni pratiche in base agli iscritti.

Table 27: Dizionario delle View - Ingredienti Pratica

5.2 Codice View

Listing 52: view.sql - Studenti Iscritti

```sql
--View necessaria per controllo posti rimanenti (in pratica)
CREATE OR REPLACE VIEW view_studenti_iscritti AS
SELECT
    p.id_pratica,
    p.corso_id,
    p.posti_totali,
    COALESCE(COUNT(ip.stud_id), 0) AS posti_occupati,
    (p.posti_totali - COALESCE(COUNT(ip.stud_id), 0)) AS posti_rimanenti
FROM pratica p
LEFT JOIN iscritto_p ip ON p.id_pratica = ip.pratica_id
GROUP BY p.id_pratica, p.corso_id, p.posti_totali;
```

Listing 53: view.sql - Ingredienti Pratica

```sql
--view per visualizzare il totale di ingredienti utilizzati per ogni pratica
--(somma degli ingredienti di tutte le ricette associate alla pratica)
--(moltiplicato per il numero di iscritti alla pratica)
CREATE OR REPLACE VIEW view_ingredienti_pratica AS
SELECT
    p.id_pratica,
    i.nome AS ingrediente_nome,
    SUM(n.quant_ing * COALESCE(NULLIF(vpp.posti_occupati, 0), 1)) AS quantita,
    i.unit_misura
FROM pratica p
JOIN pratica_svolta ps ON p.id_pratica = ps.pratica_id
JOIN ricetta r ON ps.ricetta_id = r.id_ricetta
JOIN necessita n ON r.id_ricetta = n.ricetta_id
JOIN ingrediente i ON n.ingrediente_id = i.nome
LEFT JOIN view_studenti_iscritti vpp ON p.id_pratica = vpp.id_pratica
GROUP BY p.id_pratica, i.nome, i.unit_misura;
```